Traveling Salesman Problem Heuristics and HPC

How do implementation choices and parallel execution affect solution quality under limited computational budgets?

Current milestone: shared design and a tested local software environment.

PROGRESS AT A GLANCE

What is complete—and what is not

AREA	STATUS	EVIDENCE / BOUNDARY
Scientific design	**Agreed**	Shared contracts, variants and stopping rules
Python scaffold	**Merged**	Installable package, CLI and locked dependencies
CPU image	**Built locally**	Build checks passed; PR #53 is still unmerged
SA prototype	Separate open PR	Not integrated with the agreed baseline
Comparative results	Pending	Integrated solvers and experiment runner still needed

No validated algorithm ranking or HPC speedup results yet.

ALGORITHM DESIGN · SPECIFIED

Three heuristics, one objective

For a complete, symmetric 2D TSP: visit every city once, return to the start, and minimize total tour length.

METHOD	SELECTED BASELINE	WORK BUDGET
Nearest-neighbour	Nearest unvisited city; deterministic ties	Fixed start list
Genetic algorithm	Tournament selection, ordered crossover, swap mutation, elitism	Population + generations
Simulated annealing	Segment reversal, Metropolis acceptance, geometric cooling	Proposals per chain

For symmetric TSP, SA’s four-edge cost change is O(1); applying an accepted reversal can still be O(n).

Work units differ: equal GA generations and SA proposals do not imply equal computation

Make comparisons scientifically consistent

Same objective	One distance convention per instance; always include the closing edge.
Same data rules	`float64` coordinates; `int64` city permutations with implicit closure.
Explicit search	Recorded seeds, parameters and actual work; initialization is counted.
Independent checks	Validate the final permutation and recompute its complete cost.

```
solve(instance, *, seed, budget, parameters,
      record_history=False) → SolverResult
```

IMPLEMENTATION · MERGED FOUNDATION

A small, testable numerical software stack

COMPONENT	IMPLEMENTED TODAY
Python package	Installable `tsp_hpc`; CLI help and version commands
NumPy + Numba	Typed-array smoke test with native compilation and execution
uv + lockfile	Locked dependencies; separate optional analysis environment
pytest	Package, command-line and numerical-stack checks

> **Next:** our own lightweight runner for seeds, timing, validation and results. CI automation is also pending.

The CPU image built successfully

10 / 10

packaged tests passed

Package and CLI	**6**
Definition regressions	**3**
NumPy / Numba kernel	**1**

Additional build-time checks

- Serial and parallel Numba kernels compile and execute.
- Native cache writes and JSON report write/read-back succeed.
- SIF image and SHA-256 sidecar are created.

> **Correctness, not performance:** one Numba worker; no multicore scaling or cluster compatibility claim.

Owner-provided build log · source `f6e1fcd` · Linux x86_64 · Python 3.12.16

From a source snapshot to cluster execution

Source snapshot	→	Local build	→	Transfer	→	Runtime check
Git revision		Immutable SIF		`rsync` over SSH		Slurm allocation
Dependency lock		Build provenance		Image + sidecar		Explicit mounts
Pinned base image		Artifact checksum		Verify SHA-256		JIT + CPU affinity
Recorded		**Build checks passed**		**Procedure documented**		**Aion / Iris: pending**

> Keep inputs, outputs and writable caches outside the image. Record both the source revision and the exact SIF checksum.

Aion/Iris access and storage are owner-confirmed; runtime compatibility is not yet verified.

EXPERIMENTAL METHOD · PLANNED

Separate speed, quality and time-to-target

QUESTION	CONTROLLED COMPARISON	REPORT
Implementation speed	Same variant, work, seeds and searches; same CPU architecture	Runtime Speedup $S_p = T_1 / T_p$
Solution quality	Fixed wall-clock budget; stated resources and repeated trials	Tour length Variation across runs
Time-to-target	Predefined quality threshold; retain runs that miss it	Time to target Success rate

Use shared instances and exact small-case references. Separate preprocessing and JIT warm-up from search; also report end-to-end time.

Parallelism follows each algorithm

METHOD	INITIAL SOURCE OF CONCURRENCY
Nearest-neighbour	Construct tours from a fixed set of starting cities
Genetic algorithm	Evaluate individuals or construct offspring independently
Simulated annealing	Run a fixed collection of independent chains

> Changing worker count must not increase total searches or search budgets.

Also profile: dense distances ($8n^2$ bytes) versus on-demand computation.
MPI, GPU and deeper memory optimizations remain evidence-driven options.

NEXT MILESTONES

Next: one complete, reproducible experiment

01	**Build**	CI and shared instance, tour and cost utilities.
02	**Integrate**	Serial NN, GA and SA with our seeded experiment runner.
03	**Validate**	Finished SIF, runtime mounts and Aion/Iris compatibility.
04	**Measure**	Reference quality, serial profiles, then controlled multicore scaling.

> **Takeaway:** the design and local execution foundation are in place; the next deliverable is validated solver output.